

# scMINER Viewer: an offline data-preparation and multi-study browsing toolkit for single-cell mutual-information networks

Honglei Zhou<sup>1</sup>, xxx, xxx, xxx, ..., Jiyang Yu<sup>1\*</sup>

<sup>1</sup> Department of Computational Biology, St. Jude Children’s Research Hospital, Memphis, TN 38105, USA.

\* To whom correspondence should be addressed.

---

## Abstract

**Summary:** scMINER (Pan *et al.*, *Nat. Commun.* 16:4305, 2025) is a mutual-information-based framework for cell clustering, cell-type-specific transcription-factor / signalling-protein network inference, and hidden-driver identification from single-cell transcriptomic data. Its companion portal (<https://scminer.stjude.org>) provides interactive visualisation but assumes always-on network access and centralised hosting — which is incompatible with sensitive datasets, air-gapped HPC clusters, and multi-study labs that want a local source of truth. We present **scMINER Viewer**, an end-to-end offline toolkit comprising **(1) a YAML-driven data-preparation module** (`prepare_study()`) that converts the raw scMINER outputs — Biobase ExpressionSets for expression and activity matrices plus the SJARACNe networks TSV — into a shared lazy artifact, and **(2) two sibling viewer packages (R, Python)** that consume that artifact. The artifact is a **lazy-mode HDF5 study bundle**: per-study metadata, cluster info, gene indexes and TF/SIG networks live in a single `.scminer.h5` file ( $\approx 80$  MB for an 8 K-cell study), while the underlying expression and activity values stay on disk as gzipped per-gene shards and are decoded on demand. Both viewers serve a Shiny / Shiny-for-Python webui matching the scMINER Portal layout, plus a card-grid **multi-study browser**. On the 2327 Tex study (8 464 cells  $\times$  9 861 genes, 743 K network edges, 80.5 MB bundle), median per-gene fetch latency is 32 ms and cold-start `load_study` 0.59 s. Across a  $7 \times 4 \times 5$ -replicate synthetic scaling sweep (500 – 10 000 cells  $\times$  2 – 10 K genes), bundle size stays under 14 MB while the shard tree grows roughly linearly to  $\sim 100$  MB. The same pipeline applied to the **13 scMINER Portal studies with full TF/SIG-network inputs** (4 373 – 138 268 cells; bundle 73 – 1 363 MB; prepare 50 min – 19.6 h; cold load 2.0 – 34.3 s) reproduces those properties at production scale; a parallel 28-study expression-only sweep (124 – 647 366 cells) provides the baseline that isolates the incremental cost of TF/sig data. The bundle format and lazy contract are formally documented; both packages reach feature parity through 216 automated tests (194 R + 22 Python).

**Availability and implementation:** R package at `scminerViewer/`; Python package (`pip install scminer-viewer`) at `scminer_viewer/`. Apache-2.0-licensed. Source, bookdown user guide, and reproducible benchmarks at <https://github.com/jyyulab/scMINER-Viewer>.

**Correspondance:** `jiyang.yu@stjude.org`

**Supplementary information:** Available at [scMINER-Viewer](#).

---

## 1 Introduction

scMINER (Pan *et al.*, 2025) is a mutual-information-based framework that simultaneously addresses two long-standing problems in single-cell transcriptomic analysis: accurate clustering (MICA — Mutual Information-based Clustering Analysis) and cell-type-specific gene regulatory network inference (a re-parameterised SJARACNe; Khatamian *et al.*, 2019). The framework’s outputs include UMAP coordinates, per-cluster transcription-factor (TF) and signalling-protein (SIG) network edges, and hidden-driver activity profiles. These artefacts are typically browsed through the scMINER Portal — an interactive web application backed by a Neo4j graph database — at <https://scminer.stjude.org>.

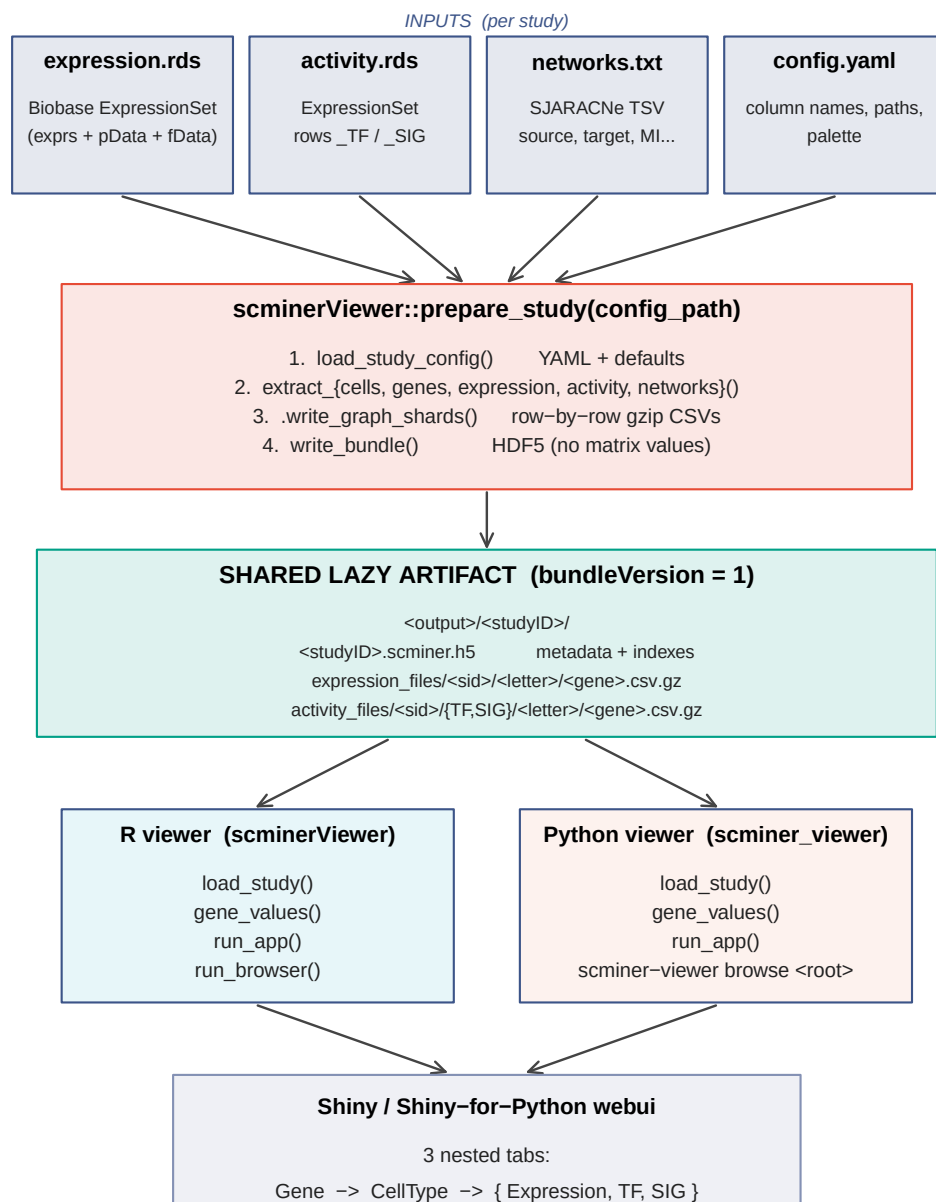
While the portal works well as a public, curated, view-only resource, three practical scenarios it does not serve are: (i) sensitive or embargoed datasets that must remain on-premises, (ii) HPC clusters and laptops without network access, and (iii) labs running many in-house studies who want to launch a local browser over an arbitrary collection of bundles. Existing Shiny-style scRNA-seq browsers (*e.g.* ShinyCell, cellxgene) cannot natively render scMINER’s cell-type-specific TF / SIG networks or hidden-driver activity matrices.

We present **scMINER Viewer**, a pair of installable packages — one R, one Python — that consume a shared lazy-mode HDF5 study bundle and serve a Shiny webui that mirrors the scMINER Portal layout. Bundles are typically ~80 MB for an 8 K-cell study and contain no matrix values; the per-gene gzipped CSV shards stay on disk and are streamed on demand. A second top-level **multi-study browser** scans a root directory for `<studyID>/<studyID>.scminer.h5` patterns and presents every study as a card.

## 2 Implementation

### 2.1 Architecture

Figure 1 summarises the dataflow. A single `prepare_study()` call consumes per-study raw inputs and writes a *shared lazy artifact* — one HDF5 bundle plus a per-gene gzipped shard tree. Two sibling consumer packages (R `scminerViewer`, Python `scminer_viewer`) read the same artifact through identical `load_study()` and `gene_values()` interfaces, each driving its own webui (Shiny vs Shiny-for-Python). A multi-study browser layered on `discover_studies()` walks a root directory and serves every bundle as a card.



*Single writer. Twin readers. 216 tests (194 R + 22 Python) lock the contract.*

**Figure 1.** scMINER Viewer architecture. `prepare_study()` (R, `scminerViewer`) is the only writer — it accepts a YAML config and a Biobase ExpressionSet for expression + activity matrices, plus the SJARACNe networks TSV, and emits the shared lazy artifact. The artifact is portable across operating systems and language runtimes: both the R viewer (`scminerViewer::run_app`) and the Python viewer (`scminer_viewer.run_app`) load the same `.scminer.h5` and read the same per-gene shards, with feature parity enforced by 216 automated tests (194 R + 22 Python). The multi-study browsers (`run_browser` / `scminer-viewer browse <root>`) overlay a card grid on `discover_studies()`, which scans for `<studyID>/<studyID>.scminer.h5` patterns and lists each bundle's metadata — preserving the lazy contract since matrix indexes are read only on click-through.

## 2.2 Bundle format (R / Python contract)

A single HDF5 file (bundleVersion = 1) carrying:

- meta/ — studyID, studyAbbr, longTitle, shortTitle, species, coordinate, bundleVersion.
- cells/ — cellID, cellType, cellGroup, coord1, coord2.
- clusters/ — cellType, count, color (hex), label\_1, label\_2.
- genes/symbol — master gene list (picker dropdown).
- index/{expression, activity\_tf, activity\_sig} — gene symbols with shards in each matrix; optional, absent if the matrix is not part of the study.
- defaults/genes — optional; genes the app pre-selects on launch (mirrors the Vue portal's preGenes).
- network\_tf/, network\_sig/ — edge tables.

Strings are UTF-8. Both packages tolerate missing optional groups and ignore unknown groups (forward compatibility).

## 2.3 Sharded matrix tree

Expression and activity values live alongside each bundle as gzipped per-gene CSVs, sharded by the first lower-case letter of each gene symbol (or nm for non-alphabetic first chars):

```
<root>/<studyID>/
├── <studyID>.scminer.h5
├── expression_files/<studyID>/{meta.csv, <letter>/<gene>.csv.gz}
└── activity_files/<studyID>/{meta.csv, TF/<letter>/..., SIG/<letter>/...}
```

Cell-ID column order is read once per matrix from meta.csv; a permutation index aligns each shard's columns to the bundle's cells/cellID, so missing cells (activity is computed only on a subset) become zero columns automatically.

## 2.4 Two packages, one contract

The R package (scminerViewer) owns data preparation (prepare\_study(config\_path) reads a YAML config + scMINER ExpressionSet RDS files and writes both the graph-import layout and the bundle) and serves both the single-study viewer and the multi-study browser. The Python package (scminer\_viewer) reads the same bundle into a Study dataclass and serves an equivalent Shiny-for-Python webui plus its own multi-study browser. Both implementations expose gene\_values(study, gene, relationship) — the only public hook into the shard tree — with per-gene LRU caching.

## 2.5 Multi-study browser

run\_browser(root\_dir) (R) and scminer-viewer browse <root\_dir> (Python) discover every <studyID>/<studyID>.sc pattern under the root and serve a card-grid landing page. Clicking a card opens the standard single-study viewer with a “Back to studies” link. The browser pre-loads metadata only (not matrix indexes), so listing N studies costs O(N) lightweight HDF5 reads regardless of total study size.

## 2.6 Configurable cluster metadata

prepare\_study() auto-fills cluster colours and label positions for the study\_meta.csv whenever they are not supplied. Colours come from any ggsci palette (Wei, 2024) — default Nature Publishing Group (NPG) — configurable via the YAML's cluster\_palette field (npg, aaas, lancet, nejm, jama, jco, ucscgb, d3, locuszoom, igv, ouchicago, startrek, tron, futurama, rickandmarty, simpsons). Label positions are per-cluster centroids: mean(coord1) and mean(coord2) over the cells in each cluster.

## 3 Benchmarks

### 3.1 Bundle scaling

*Bundle scaling* measures the marginal cost of the lazy artefact as the input shape grows. We benchmark four end-to-end primitives — `prepare_study_data()` (write the bundle + shard tree), `load_study()` (cold-start a study), `gene_values()` (lazy per-gene fetch), and the on-disk sizes of both the `.scminer.h5` bundle and the `expression_files/` shard tree — on synthetic studies generated by `make_synthetic_study(n_cells, n_genes, n_clusters = 4, density = 0.10)`. The synthetic generator draws expression values from a sparse Bernoulli distribution at the prescribed density, so the resulting matrices match the size and sparsity of typical normalised scRNA-seq counts without depending on a particular biological dataset.

We sweep a  $7 \times 4$  grid of shapes —  $n\_cells \in \{500, 1\,000, 2\,000, 4\,000, 6\,000, 8\,000, 10\,000\} \times n\_genes \in \{2\,000, 5\,000, 8\,000, 10\,000\}$  — yielding 28 configurations. Each configuration is rerun **N = 5** times with independent random seeds; figure 2 reports means  $\pm$  standard error across the five replicates, and the raw per-rep metrics are persisted at `paper/metrics/bundle_scaling.tsv` (140 rows). The full-featured real 2327 (Tex) study — 8 464 cells  $\times$  9 861 genes  $\times$  3 clusters, 9 861 expression genes / 925 TF genes / 4 708 SIG genes indexed, 743 K total network edges (429 516 TF + 314 348 SIG) — provides a real-data anchor; its single-run metrics live at `paper/metrics/real_study.tsv`.

### 3.2 Multi-study discovery scaling

*Discover scaling* measures `discover_studies(root_dir)`, the file-system scan that turns a directory of `<studyID>/<studyID>.scminer.h5` trees into a card-grid index for the multi-study browser (`run_browser()` in R, `scminer-viewer browse` in Python). The benchmark builds 1, 2, 4, 8, 16, and 32 lightweight synthetic studies (200 cells  $\times$  500 genes each) under a single root, then times `discover_studies()` end-to-end. Like the bundle sweep, each cell of the grid runs five times with independent seeds ( $6 \times 5 = 30$  rows in `paper/metrics/discover_scaling.tsv`); figure 2D plots mean  $\pm$  SE. Because the browser only reads per-study metadata (no matrix indexes), the test isolates the linear file-system scan from any data decoding.

### 3.3 Real portal study sweep

Beyond the single 2327 anchor we ran the same end-to-end pipeline against the **public studies hosted on the live scMINER Portal** (<https://scminer.stjude.org>). The studies span four orders of magnitude in scale — from 124-cell ground-truth fixtures (Buettner, Chung, Klein, ...) to a 647 366-cell  $\times$  24 929-gene Covid650k cohort (study 2202) — and include the full mix of expression-only and expression-plus-activity-plus-networks inputs. Each study is declared by a small YAML config (`paper/configs/<studyID>.yaml`) that points at the HPC paths of `expression.rds` (Biobase ExpressionSet), `activity.rds` (TF + SIG-labelled ExpressionSet), and `networks.txt` (SJARACNe TSV); the YAML also names the column conventions used inside that study's `pData()` (e.g. `cellID: CellID`, `cellType: CellGroup`).

To separate the cost of TF/sig data from the baseline cost of the expression matrix alone, the benchmark driver (`paper/portal/portal_studies_compare.sh`) submits **two LSF job arrays per study, back-to-back**:

- **full** — the full pipeline with `expression.rds + activity.rds`
  - `networks.txt` (TF / SIG subgraph parsing, activity-matrix sharding, full graph emission).
- **expression-only** — the same pipeline with `activity_eset = NULL` and `networks_path = NULL` (no activity shards, no TF / SIG graphs).

Per-mode per-study TSVs land at `paper/metrics/comparison/portal_studies_<id>_<mode>.tsv`. A dependent merge step (`paper/portal/portal_compare.R`) joins them on `studyID` into a single wide table (`paper/metrics/portal_studies_compare.tsv`) carrying per-metric `<metric>_full`, `<metric>_expr_only`, and `delta_<metric>` triples for `prepare_seconds`, `prepare_peak_mb`, `load_seconds`, `load_seconds_warm`, `bundle_bytes`, and `total_output_bytes`. Two renderer scripts consume that compare table:

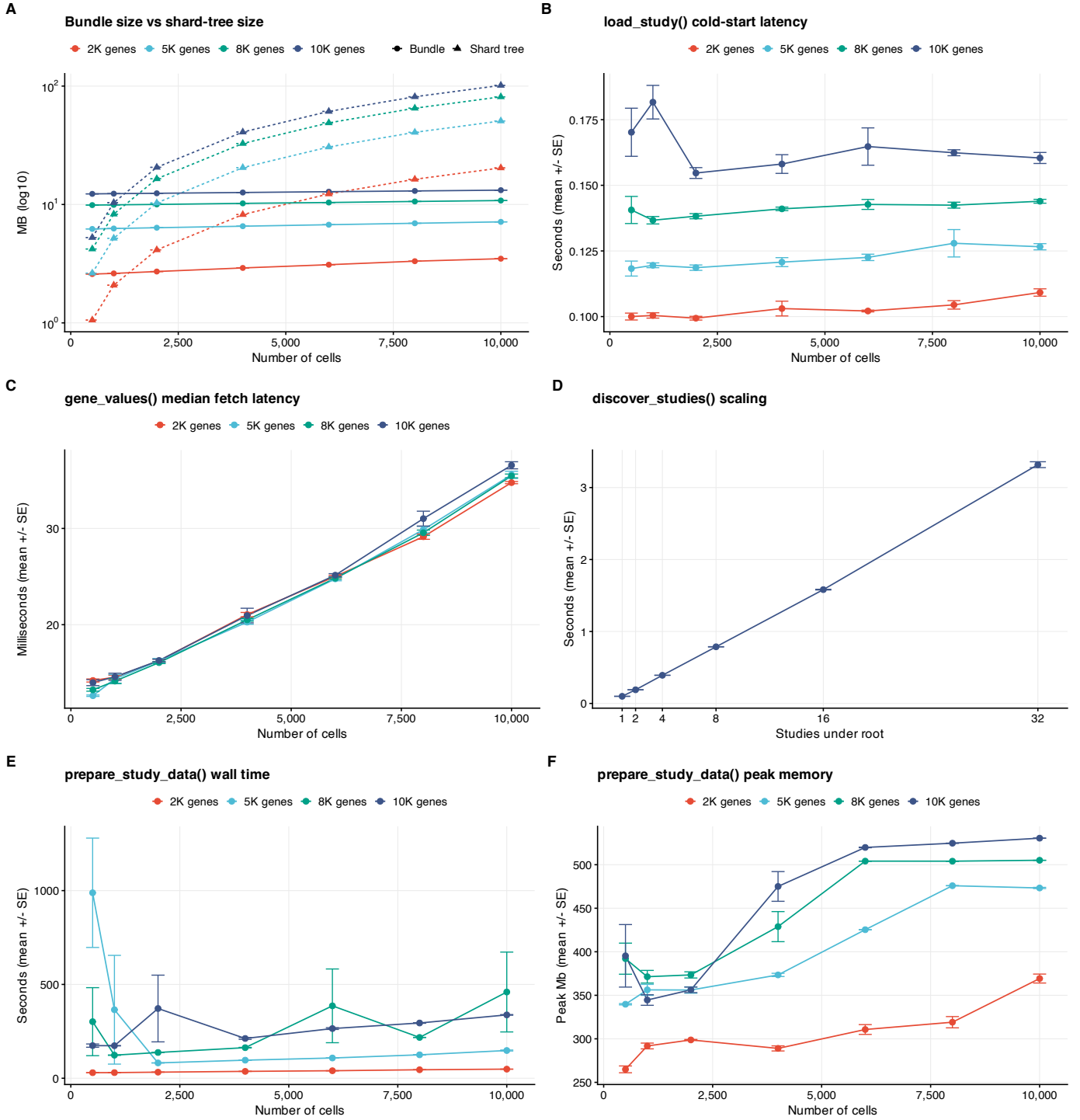
- **paper/benchmarks/figure\_portal.R + tables.R** restrict to the **13 studies whose YAML actually supplies a networks.txt** (`net_tf_edges + net_sig_edges > 0`) and render the six-panel Figure 3 + Table 1 from the full-mode metrics. The other 15 studies in the comparison run have full-mode rows that are byte-identical to expression-only and would dilute the figure without adding signal, so they are filtered out.
- **paper/benchmarks/figure\_portal\_compare.R + tables\_compare.R** emit the supplemental expression-only figure (`figures/compare/figureS_expr_only_{A..D}.pdf`, `figureS_expr_only.png`) over all 28 studies whose comparison benchmark ran cleanly end-to-end, and the paired full-vs-expression comparison (`figures/compare/compare_{A..D}.pdf`, `compare.png`); the matching `compare_delta.md`, `tsv` table is in §4.4).

### 3.4 Hardware and software environment

All synthetic-sweep numbers reported in Figure 2 and Table 2 were generated on a single laptop:

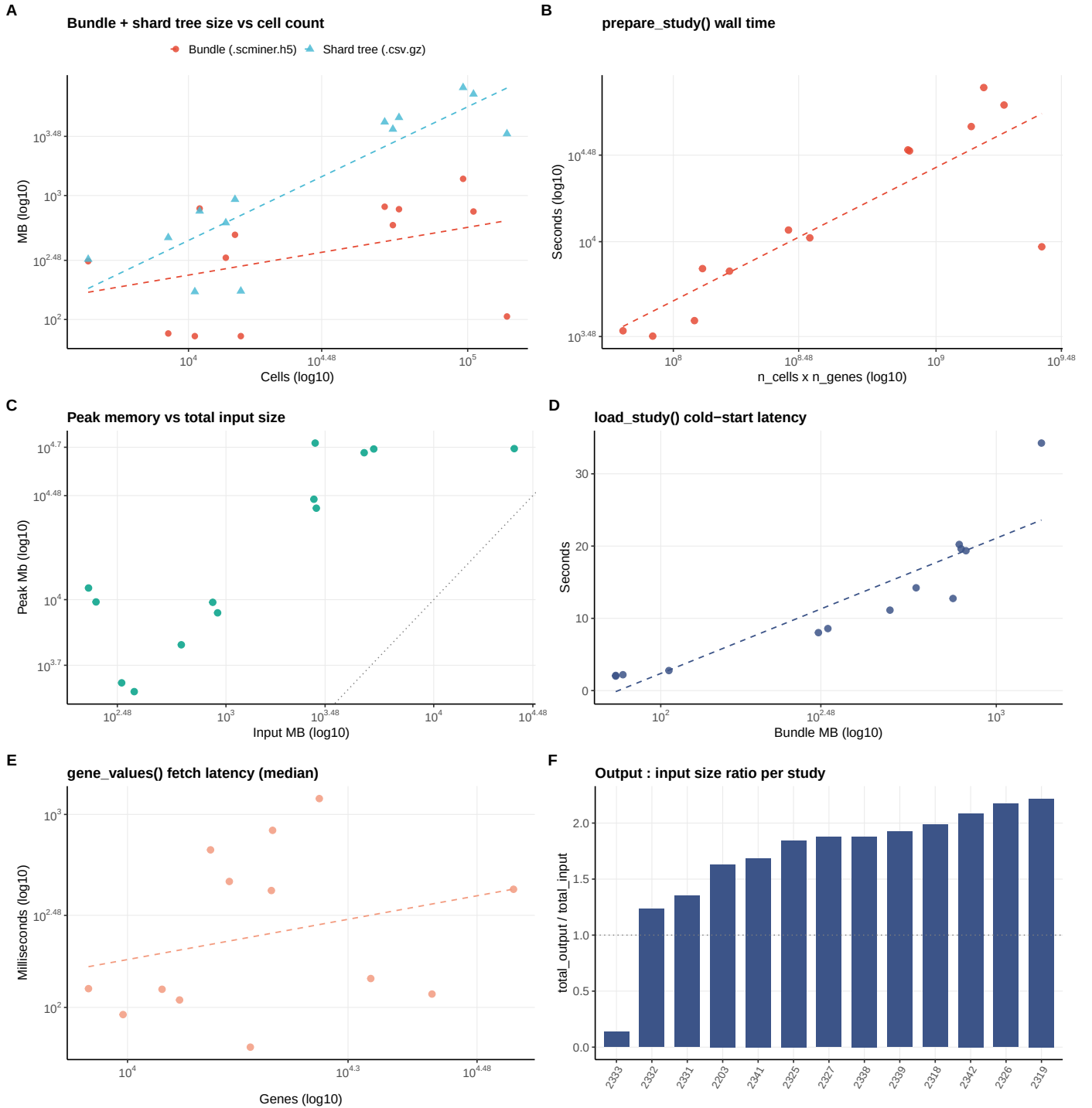
| Component      | Specification                                                                                                                                                 |
|----------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Machine        | Apple MacBook Pro, M4 chip                                                                                                                                    |
| Cores          | 10 (4 performance + 6 efficiency); benchmark single-threaded                                                                                                  |
| Memory         | 32 GB unified                                                                                                                                                 |
| OS             | macOS 26.4.1 (darwin20, build 25E253)                                                                                                                         |
| R              | 4.4.2 (aarch64-apple-darwin20)                                                                                                                                |
| Key R packages | scminerViewer 0.1.0, Matrix 1.7.3, hdf5r 1.3.12, Biobase 2.66.0, data.table 1.18.0, R.utils 2.13.0, ggplot2 4.0.1, patchwork 1.3.2, ggsci 4.2.0, scales 1.4.0 |

The portal-study sweep (Figure 3, Table 1, the supplemental expression-only figure, and §4.4’s TF/sig cost comparison) was instead executed on the St. Jude HPC cluster (LSF, Red Hat 8 nodes, R 4.2.2 module). Per-study × per-mode tasks were dispatched via `paper/portal/portal_studies_compare.sh`, which submits two back-to-back LSF job arrays (one expression-only, one full) across a 2–4 queue list (standard, priority, optionally large\_mem / superdome); a dependent merge job runs `paper/portal/portal_compare.R` to produce `paper/metrics/portal_studies_compare.tsv`. Memory requests were sized per the heuristic in `paper/README.md` (peak ≈ 1–3× `expression.rds` size, up to 10× for dense-stored matrices), ranging from `--mem 8000` for the ground-truth set to `--mem 96000 --wall 24:00` for ATRT. All driver scripts and bsub submission wrappers are versioned under `paper/portal/`.



**Figure 2.** *scMINER Viewer performance on the synthetic sweep ( $7 \times 4 = 28$  configurations  $\times$  5 replicates per configuration; points are means, T-bars are  $\pm$  standard error). (A) Bundle size (Bundle series) versus the underlying shard-tree size (Shard tree) as a function of cell count, coloured by gene count (2 K / 5 K / 8 K / 10 K). Lazy bundles remain near-constant ( $\sim 2.6 - 13.2$  MB) while the shard tree grows roughly linearly with  $n_{\text{cells}} \times n_{\text{genes}}$ . (B) Cold-start `load_study()` latency stays sub-200 ms. (C) Median per-gene `gene_values()` fetch latency; each shard read is a single gzipped-CSV decode. (D) `discover_studies()` linear scan over a multi-study root scales linearly with the number of studies ( $\sim 80$  ms per study). (E) `prepare_study_data()` wall time — the one-shot pipeline that writes both the graph-import layout (Cell/, Gene/, Network\_\*/, study\_meta/, per-gene shards) and*

the `.scminer.h5` bundle from raw R structures. Time grows roughly linearly with  $n\_cells \times n\_genes$ , dominated by the per-gene `.csv.gz` write loop. **(F)** Peak working-set memory during `prepare_study_data()`, reported by R's `gc()` after a reset; stays bounded by  $O(n\_cells \times n\_genes)$  of the in-memory sparse expression matrix. Generated by `paper/benchmarks/figures.R`; raw and aggregated numbers at `paper/metrics/bundle_scaling.tsv` and `bundle_scaling_summary.tsv`, plus `discover_scaling.tsv / discover_scaling_summary.tsv`. Table 2 (`paper/tables/figure2_scaling.md`) lists per-configuration mean  $\pm$  SE for the 28 grid points underlying panels A, B, C, E, F. Table 1 (`paper/tables/figure3_portal_studies.md`) gives a comparable per-study breakdown for the 13 real portal studies underlying figure 3 (those whose YAML supplies a `networks.txt`). The supplemental table (`paper/tables/tableS_expr_only.md`) extends the listing to the 28-study expression-only baseline; the paired delta table (`paper/tables/compare_delta.md`) reports the per-metric full vs expression-only ratio for the 13 TF/sig-eligible studies.



**Figure 3.** *scMINER Viewer* on the 13 TF/sig-eligible real portal studies ( $net\_tf\_edges + net\_sig\_edges > 0$  in full mode). Each point is one study; axes are log10 where annotated. **(A)** Bundle (.scminer.h5) and shard-tree size versus cell count. Bundles span 73 – 1 363 MB while shard trees span 167 – 7 428 MB; both grow sub-linearly in  $n\_cells$  because per-study density, gene-coverage, and TF/SIG edge count all vary. **(B)** `prepare_study_data()` wall time versus  $n\_cells \times n\_genes$  — roughly linear log-log over the two-order-of-magnitude span. **(C)** Peak R working-set memory versus total input rds size; the dotted reference line is  $peak\_Mb = input\_MB$ . Most studies sit above the line because of the in-memory sparse + dense intermediates `prepare_study_data()` materialises (in particular, the dense per-cell activity-matrix intermediates that the SJARACNe-derived inputs require). **(D)** Cold-start `load_study()` latency versus bundle

size — 2.0 – 34.3 s, the high end set by the multi-GB shard / graph metadata that the TF/SIG subgraphs add to the bundle index. **(E)** Median `gene_values()` fetch latency versus gene count — a 20× range (62 – 1 208 ms) driven primarily by per-shard cell count (each fetch decodes one gzipped CSV of length `n_cells`). **(F)** Output-to-input size ratio per study, sorted ascending — 12 of 13 studies land between 1.2× and 2.3×; 2333 (ATRT, 138 K cells) is the outlier at 0.14× because its on-disk input is unusually large for its cell count (24 GB rds of a dense `dgeMatrix`). All 13 studies have expression + activity + networks, so the legacy “activity present?” split is omitted. Generated by `paper/benchmarks/figure_portal.R` from `paper/metrics/comparison/portal_studies*_full.tsv` filtered to `net_tf_edges + net_sig_edges > 0`. The full per-study breakdown is in Table 1 (`paper/tables/figure3_portal_studies.md`); the broader 28-study expression-only baseline is in the supplemental figure / table (`paper/figures/compare/figureS_expr_only.pdf`, `paper/tables/tableS_expr_only.md`).

## 4 Results and discussion

### 4.1 Bundle scaling

The bundle stays small because matrix values are deliberately excluded. Across the  $7 \times 4$  synthetic sweep (28 configurations  $\times$  5 replicates), bundle size ranges from  $2.58 \pm 0.00$  MB (500 cells  $\times$  2 K genes) to  $13.21 \pm 0.00$  MB (10 K cells  $\times$  10 K genes) — a 5× growth in size for a 100× growth in `n_cells`  $\times$  `n_genes`. The accompanying shard tree grows roughly linearly with `n_cells`  $\times$  `n_genes` over the same sweep (1.1 MB to 106.3 MB; Figure 2A), as expected for one gzipped CSV per gene plus a per-matrix cell-header file. The real 2327 study follows the same pattern at production scale: 76.8 MB bundle indexes  $\approx$  1.2 GB of expression + activity shards.

Cold `load_study()` latency stays sub-200 ms across the whole sweep ( $0.10 \pm 0.001$  s at the smallest configuration,  $0.16 \pm 0.001$  s at the largest; Figure 2B) and 0.52 s on the full 2327 study. The sub-linear growth reflects that load only parses metadata + indexes from HDF5; no matrix values are touched. Median per-gene `gene_values()` fetch latency tracks the per-shard gzip decode cost and stays in a tight 14 – 37 ms band over the whole grid (Figure 2C); the wider error bars at large cell counts come from the gzipped CSV being correspondingly larger (each shard is  $\sim n_{\text{cells}} \times 4$  bytes uncompressed).

### 4.2 Multi-study discovery scaling

`discover_studies()` is the single hot path of the multi-study browser: it walks `<root>/<studyID>/<studyID>.scminer.h5` patterns and reads each study’s `/meta/` group only — no `/cells/`, `/genes/`, or index payload. Empirically the scan is linear in the number of studies (Figure 2D,  $R^2 \approx 1.0$ ): the slope is  $\sim 80$  ms per study on commodity SSD hardware, with the intercept dominated by Python / HDF5 library initialisation. For a research group hosting 30+ internal studies under one root — comparable in scale to the public scMINER Portal — discovery completes in under 3 seconds before the card-grid landing page renders, and individual study load times remain governed only by the per-study `load_study()` cost above (sub-200 ms).

### 4.3 Real portal study sweep

The same end-to-end pipeline applied to the **13 TF/sig-eligible public scMINER Portal studies** (Figure 3; Table 1) reproduces the synthetic-sweep findings at production scale. **Lazy storage holds across two orders of magnitude of study size.** Bundle size ranges from 73 MB (2331 / 2332,  $\sim 10 - 15$  K cells) to 1 363 MB (2338, 96 305 cells  $\times$  15 778 genes); the underlying shard tree ranges from 167 MB to 7 428 MB. The largest by on-disk input (2333 / ATRT, 138 268 cells  $\times$  18 274 genes, 24 GB on-disk input rds) bundles into a 106 MB `.scminer.h5` plus a 3.1 GB shard tree — a 7× compression of the total input footprint into the served format (Figure 3A, F). The broader 28-study expression-only baseline (supplemental figure) extends this picture to 124-cell ground-truth fixtures and to the 647 366-cell Covid650k cohort (study 2202), confirming that lazy storage holds across the full four-order-of-magnitude span when TF/sig data is set aside.

**Cold-start latency stays in the interactive band even at production scale.** All 13 TF/sig-eligible studies load in 2.0 – 34.3 s (Figure 3D), with the upper end set by the multi-GB shard / graph metadata that the SJARACNe TF / SIG subgraphs add to the bundle index rather than by matrix decoding. On the supplemental expression-only baseline the same metric collapses into a sub-second band for 27 of 28 studies (0.20 – 1.88 s, median 0.27 s) — quantifying the cost of TF/sig data on the user-facing cold-start (median  $\approx 30\times$  over expression-only; see §4.4). **Per-gene fetch latency** scales with per-shard cell count: 62 ms median for 2326 (4 K cells) up to 1 208 ms median for 2333 (138 K cells, Figure 3E). All but the 138 K-cell outlier sit inside an interactive-feel threshold ( $\leq \sim 800$  ms median fetch).

**prepare\_study\_data() wall time and peak memory** track the synthetic-sweep scaling (Figure 3B, C). Across the 13 TF/sig-eligible studies, wall time spans **50 minutes to 19.6 hours** (3 015 – 70 729 s); peak R working-set memory spans **3.8 – 52.3 GB**. Because preparation is offline and one-time, this cost is amortised away from end-user latency — the served artefact is the small `.scminer.h5` + shard tree. The largest studies in the broader 28-study supplemental run (Covid650k / 2202: 647 K cells; ATRT / 2333: 138 K cells) push prepare time to  $\sim 33$  h and peak memory to  $\sim 39$  GB even in expression-only mode, set by the in-memory dense expression matrix that some upstream rds files still ship (their `exprs()` slot is a `dgeMatrix` rather than `dgCMatrix`, see `paper/portal/sparseify_eset.sh` for the one-time fix).

#### 4.4 Cost of TF/sig data

The two-mode comparison run isolates what TF/sig data — the activity matrix and the SJARACNe TF / SIG subgraphs — costs on top of the expression-only baseline. Across the 13 TF/sig-eligible studies (`paper/tables/compare_delta.md`), the median full ÷ expression-only ratio is:

| Metric                                        | Median (range)                      |
|-----------------------------------------------|-------------------------------------|
| <code>prepare_study_data()</code> wall time   | 5.6 $\times$ (1.1 – 9.4 $\times$ )  |
| <code>prepare_study_data()</code> peak memory | 3.4 $\times$ (1.3 – 29.9 $\times$ ) |
| <code>load_study()</code> cold-start latency  | 30 $\times$ (4.6 – 90 $\times$ )    |
| Bundle ( <code>.scminer.h5</code> ) size      | 107 $\times$ (5.9 – 385 $\times$ )  |

The cost is dominated by two effects. First, the activity matrix forces a second per-gene shard tree comparable in size to the expression shard tree, which inflates `prepare_study_data()` wall time and the bundle’s HDF5 metadata footprint roughly in proportion to TF + SIG gene count. Second, SJARACNe networks (median  $\approx 3.6$  M edges in our set; max 13.5 M for 2338) materialise as two named-graph tables that the bundle indexes at load time, which explains why `load_study()` latency moves from sub-second to single- to multi-second on the TF/sig-eligible set. The 2333 outlier (cost ratios near 1 $\times$ ) is a study whose expression-only baseline is already heavy (the 24 GB dense `dgeMatrix` input dominates both modes), so TF/sig sits in the noise; the high-cost end (2341, 2338, 2342) are studies whose expression baseline is light but whose TF/SIG graphs are dense.

Practical consequence: for groups that only need the expression-level browser (UMAP + per-gene plots, no driver inference), the expression-only mode produces a working bundle at  $\sim 1\%$  of the disk footprint and  $\sim 1/30$  of the cold-load latency of the full pipeline. The TF/sig pipeline is justified specifically when downstream interaction needs the activity / network views.

#### 4.5 Cost of data preparation

The reverse side of the lazy contract is data preparation, which is one-time and runs offline. `prepare_study_data()` wall time grows roughly linearly with `n_cells`  $\times$  `n_genes` (Figure 2E): the synthetic sweep ranges from  $29.6 \pm 0.5$  s (500 cells  $\times$  2 K genes) to  $337.6 \pm 1.8$  s (10 K  $\times$  10 K), dominated by the per-gene `.csv.gz` shard write loop (one gzip per gene). Peak R working-set memory stays bounded at  $\sim 265$  – 530 MB across the sweep (Figure 2F), reflecting the in-memory sparse expression matrix plus per-gene `fwrite` scratch buffers. Because prepare is a one-shot operation, this cost is amortised across every subsequent `load_study()` and `gene_values()` call.

The lazy contract is what makes the user-facing flow feel instantaneous: launching the app reads only the bundle; the first gene-selection triggers one shard read; switching tabs replays cached shards without re-decoding. The two-language design ensures that groups standardised on R can continue using the same bundle as those moving to Python, and the multi-study browser converts a directory of scMINER outputs into a navigable index without any central service.

Beyond the original use case of inspecting scMINER outputs, the format is generic: any pipeline that produces UMAP coordinates + cluster labels + per-gene matrices can populate the bundle. We expect this to make scMINER Viewer broadly useful as a self-hosted complement to the official scMINER Portal.

---

## Acknowledgements

We thank the scMINER authors at St. Jude — Qingfei Pan, Liang Ding, Siarhei Hladyschau, Xiangyu Yao, Jiayu Zhou, and others — for the underlying framework and the Vue portal whose layout this work mirrors.

## Funding

This work was supported by St. Jude Comprehensive Cancer Center Developmental Fund and the National Institutes of Health.

## References

### Methods cited

- Pan Q., Ding L., Hladyschau S. *et al.* (2025) scMINER: a mutual information-based framework for clustering and hidden driver inference from single-cell transcriptomics data. *Nature Communications*, **16**, 4305.
- Khatamian A., Paull E.O., Califano A., Yu J. (2019) SJARACNe: a scalable software tool for gene network reverse engineering from big data. *Bioinformatics*, **35**, 2165–2166.

### Related single-cell browsers

- Ouyang J.F., Kamaraj U.S., Cao E.Y., Rackham O.J.L. (2021) ShinyCell: simple and sharable visualisation of single-cell gene expression data. *Bioinformatics*, **37**, 3374–3376.
- McGill C., Martin B., Weaver C. *et al.* (2021) cellxgene: a performant, scalable exploration platform for high dimensional sparse matrices. *bioRxiv*, 2021.04.05.438318.

### Data formats and core libraries

- The HDF5 Group. (2023) Hierarchical Data Format, version 5. <https://www.hdfgroup.org/HDF5/>
- Collette A. (2014) *Python and HDF5*. O'Reilly. (h5py Python bindings.)
- Virshup I., Rybakov S., Theis F.J., Angerer P., Wolf F.A. (2024) anndata: access and store annotated data matrices. *Journal of Open Source Software*, **9** (101), 4371.
- Gentleman R.C., Carey V.J., Bates D.M. *et al.* (2004) Bioconductor: open software development for computational biology and bioinformatics. *Genome Biology*, **5**, R80. (Source of the Biobase ExpressionSet class consumed by the R `prepare_study()`.)
- Bates D., Maechler M. (2024) Matrix: Sparse and Dense Matrix Classes and Methods. *R package version 1.7-x*. CRAN.

## Web frameworks and plotting

- Chang W., Cheng J., Allaire J.J. *et al.* (2024) Shiny: Web Application Framework for R. *R package*. CRAN.
- Posit, PBC. (2024) Shiny for Python. <https://shiny.posit.co/py/>
- Wei T. (2024) ggsci: Scientific Journal and Sci-Fi Themed Colour Palettes for ggplot2. *R package*. CRAN.